

Model Router — Database Architecture

Database explanation for the Prisma 7 + PostgreSQL schema. The design covers authentication, persistent sessions, AI provider access, model routing, conversations, context switching, token usage, cost and request tracking.

1. High-Level Architecture

```
USER
|
|-- Authentication
|   |-- AuthAccount (Google / OAuth)
|   |-- Session (persistent login)
|
|-- ProviderAccount (user's AI provider access)
|
|-- Conversation
|   |-- Message --> Model
|   |-- Request  --> Model
|
|-- Request --> Model --> Provider
```

2. Tables at a Glance

Table	Purpose
User	One record for each application user.
AuthAccount	External login identities such as Google.
Session	Persistent browser/device login sessions.
Provider	Global AI providers such as OpenAI, Anthropic or Google.
ProviderAccount	A user's credentials/access for a specific provider.
Model	AI model registry, capabilities, context limits and pricing.
Conversation	A user's chat; deliberately not tied to one model.
Message	Actual visible conversation messages.
Request	Router execution, optimization, model selection, usage and cost.

3. User

The central identity table. One row represents one application account.

Field	Type	Purpose
id	UUID	Primary key
username	String	Unique username; supports username/password login
email	String	Unique email; supports email/password login
phone	String?	Optional unique phone number
passwordHash	String?	Hashed local password; null for OAuth-only users
firstName	String?	Optional profile name
lastName	String?	Optional profile name
emailVerifiedAt	DateTime?	Email verification timestamp

Field	Type	Purpose
phoneVerifiedAt	DateTime?	Phone verification timestamp
isActive	Boolean	Account status
createdAt / updatedAt	DateTime	Lifecycle timestamps

4. AuthAccount

Separates external authentication identities from the User record.

Field	Type	Purpose
id	UUID	Primary key
userId	UUID	Foreign key to User
provider	String	Example: google, github, microsoft
providerAccountId	String	External provider's unique account ID
createdAt / updatedAt	DateTime	Lifecycle timestamps

Example: one User can have a Google AuthAccount and later a GitHub AuthAccount without changing the User table.

5. Session

Provides persistent login across browser/device visits.

Field	Type	Purpose
id	UUID	Primary key
userId	UUID	Foreign key to User
sessionToken	String	Unique session credential/hash
ipAddress	String?	Optional session/device information
userAgent	String?	Optional browser/device information
expiresAt	DateTime	Session expiration
createdAt / updatedAt	DateTime	Lifecycle timestamps

One user can have multiple sessions: for example, laptop + phone + desktop. The browser should use a secure HTTP-only cookie. In production, store a hash of the session token rather than the raw token.

6. Provider

Global registry of AI companies/providers.

Field	Type	Purpose
id	UUID	Primary key
name	String	Unique provider identifier
displayName	String	Human-readable provider name
isActive	Boolean	Whether the provider is currently available
createdAt / updatedAt	DateTime	Lifecycle timestamps

7. ProviderAccount

Connects a user to an AI provider and stores encrypted access credentials.

Field	Type	Purpose
id	UUID	Primary key
userId	UUID	Foreign key to User
providerId	UUID	Foreign key to Provider
encryptedCredentials	String	Encrypted provider credentials; never raw API keys
isActive	Boolean	Whether this provider connection is usable
createdAt / updatedAt	DateTime	Lifecycle timestamps

The unique constraint (userId, providerId) means one user has one provider account/credential record per provider.

8. Model

Global registry of individual AI models. The router uses this information to filter and rank models.

Field	Type	Purpose
id	UUID	Primary key
providerId	UUID	Foreign key to Provider
name	String	Provider's model identifier
displayName	String	Human-readable model name
contextWindow	Int	Maximum context capacity
supportsVision	Boolean	Supports image/vision input
supportsToolCalling	Boolean	Supports tool/function calling
supportsStructuredOutput	Boolean	Supports structured output
supportsReasoning	Boolean	Supports reasoning capability
inputCostPerMillionTokens	Decimal	Input pricing
outputCostPerMillionTokens	Decimal	Output pricing
isActive	Boolean	Whether model can be selected

Routing example: if a prompt requires vision, the router can first filter models where `supportsVision = true`, then compare cost, context capacity and other requirements.

9. Conversation

Represents a chat owned by a user. It is intentionally NOT tied to a single model.

Field	Type	Purpose
id	UUID	Primary key
userId	UUID	Foreign key to User
title	String?	Optional chat title
createdAt / updatedAt	DateTime	Chat timestamps

This is the key design choice for context switching. One conversation can contain messages generated by different models.

Context Switching Example

```
Conversation #123
■■■ User message
■■■ Assistant message → Model A
■■■ User message
■■■ Assistant message → Model B
■■■ User message
■■■ Assistant message → Model C
```

10. Message

Stores the actual conversation content shown to the user.

Field	Type	Purpose
id	UUID	Primary key

Field	Type	Purpose
conversationId	UUID	Foreign key to Conversation
role	Enum	user, assistant, system or tool
content	String	Message content
modelId	UUID?	Model that generated an assistant message; null for user/system messages
inputTokens	Int?	Token usage associated with the message
outputTokens	Int?	Generated token count
totalTokens	Int?	Total token count
createdAt	DateTime	Message timestamp

11. Request

Records what the Model Router did for one model invocation. It is separate from Message.

Field	Type	Purpose
id	UUID	Primary key
userId	UUID	User making the request
conversationId	UUID	Conversation being continued
modelId	UUID	Model selected by the router
originalPrompt	String	Original user prompt
optimizedPrompt	String?	Prompt after optimization
contextTokensBefore	Int?	Context size before compression/optimization
contextTokensAfter	Int?	Context size after optimization
contextWasCompressed	Boolean	Whether context compression occurred
inputTokens	Int?	Provider-reported input tokens
outputTokens	Int?	Provider-reported output tokens
totalTokens	Int?	Total token usage
latencyMs	Int?	Request latency
cost	Decimal?	Estimated/actual request cost
status	Enum	pending, processing, completed or failed
errorMessage	String?	Failure details when applicable
createdAt	DateTime	Request timestamp

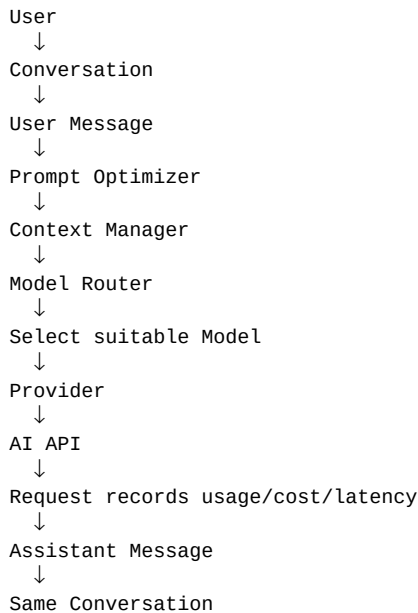
12. Why Message and Request are Separate

Message answers: “What does the user see in the conversation?”

Request answers: “What did the router do to produce this response?”

A request can therefore record the original prompt, optimized prompt, context compression, selected model, token usage, latency and cost without polluting the conversation message itself.

13. End-to-End Request Flow



14. Key Relationships

Relationship	Meaning
User 1:N AuthAccount	One user can have multiple OAuth identities.
User 1:N Session	One user can stay logged in on multiple devices.
User 1:N ProviderAccount	A user can connect multiple AI providers.
Provider 1:N Model	A provider offers many models.
User 1:N Conversation	A user can have many chats.
Conversation 1:N Message	A chat contains many messages.
Conversation 1:N Request	A chat can generate many model requests.
Model 1:N Message	A model can generate messages across many chats.
Model 1:N Request	A model can process many routed requests.

15. Design Principles

- **Conversation is model-agnostic:** this enables context switching between models in one chat.
- **User is the identity:** login methods and external providers are separated from the core user record.
- **Sessions are separate:** multiple devices can remain authenticated independently.
- **Provider and Model are separate:** one provider can expose many models.
- **ProviderAccount is user-specific:** it represents the AI access available to a particular user.
- **Message and Request are separate:** conversation content is separated from routing telemetry and cost data.
- **Credentials are encrypted:** raw AI API keys should never be stored in plaintext.

This document describes the current database architecture for the Model Router MVP. Additional domains such as billing/subscriptions, tool calls, attachments, model-specific routing policies, and detailed context snapshots can be added later without changing the core authentication and conversation design.